ARACHIND

Orin AI Testbed

System Documentation & Cursor Agent Prompt

Hardware	NVIDIA Jetson Orin Nano Super (ARM64, 8GB, CUDA 12.6)
Network	Tailscale mesh — 100.68.165.28
Hostname	dog-station
OS	Ubuntu 22.04, JetPack 6 (L4T R36.4.7)
Team	Ben (Infrastructure) / Priyan (ML Development)

Contents

Part 1: System Documentation

Why This Exists

Manual SSH, manual Docker commands, manual model reloads -- none of that scales when you're iterating on ML pipelines with live sensor input. This testbed gives you:

- **One-click container management** via Portainer (no SSH needed for 90% of ops)
- **Live system telemetry** via Heartbeat dashboard (GPU thermals, memory pressure, container health)
- **Remote desktop** embedded directly in the monitoring dashboard (no separate VNC client)
- **Crash isolation** -- develop on your Mac, containers run on the Orin. If a model OOMs, your editor doesn't die
- **Tailscale mesh** -- no port forwarding, no firewall rules, no VPN configs. Just works from anywhere

Bottom line: You never need to SSH into the Orin for normal development. Push code, rebuild containers, monitor everything -- all from Cursor on your Mac.

Hardware

Component	Spec
Board	NVIDIA Jetson Orin Nano Super Developer Kit
CPU	6-core ARM Cortex-A78AE
GPU	1024-core Ampere (CUDA 12.6)
RAM	8 GB LPDDR5 (shared CPU/GPU)
Storage	234 GB NVMe (~150 GB free)
JetPack	6.0 (L4T R36.4.7)
OS	Ubuntu 22.04
Network	Tailscale mesh, local WiFi
Planned sensor	Intel RealSense D455i (RGB + stereo depth + IMU)

Memory is shared -- GPU VRAM comes out of the same 8 GB. Watch the Heartbeat dashboard when loading models. If you cross ~6.5 GB total, OOM-killer will strike.

Connections & Credentials

SSH Access

Field	Value
Host	100.68.165.28 (Tailscale)
User	dog
Password	r00t
Port	22 (default)
Command	sshpass -p 'r00t' ssh dog@100.68.165.28

Web Services

Service	URL	Auth
Portainer	https://100.68.165.28:9443	admin / vaqfoz-komrip-2Fiqjy
Heartbeat	http://100.68.165.28:8888	None (system monitor + VNC)
TankSense	http://100.68.165.28:7860	None (ML model demo)
noVNC	http://100.68.165.28:6080/vnc.html	VNC password: r00t
VNC Direct	100.68.165.28:5901	Password: r00t

Portainer value over CLI: Visual container logs, one-click restart, resource graphs, image management, volume inspection -- no SSH session needed.

Running Services

Container	Port	Purpose	Restart Policy
heartbeat	8888	System telemetry + VNC viewer	unless-stopped
tanksense	7860	ML inference (tank detection)	unless-stopped
portainer	9443	Docker management GUI	always
novnc-proxy	6080	Browser VNC via websockify	unless-stopped

All containers auto-restart on reboot. VNC server (TigerVNC) runs on the host, not in Docker.

Directory Structure (on Orin)

```
/home/dog/

+-- heartbeat/
```

```
| +-- app.py (FastAPI + WebSocket monitor backend)
| +-- index.html (Dashboard frontend: CPU/GPU/Docker/VNC)
| +-- Dockerfile
+-- novnc-proxy/
| +-- Dockerfile (websockify + noVNC static files)
+-- tanksense-demo/
| +-- app.py (FastAPI inference server)
| +-- index.html (HF Space frontend)
| +-- requirements.txt
| +-- Dockerfile
+-- (future: realsense-pipeline/)
```

Development Workflow

The Golden Loop

- 1. Edit code in Cursor (on your Mac)
- 2. SCP to Orin: sshpass -p 'r00t' scp ./app.py dog@100.68.165.28:/home/dog/project/
- 3. Rebuild + redeploy via SSH or Portainer
- 4. Test in browser (model endpoint or Heartbeat dashboard)
- 5. Check GPU/memory in Heartbeat: http://100.68.165.28:8888

Why Not Develop Directly on the Orin?

- 8 GB shared memory -- your IDE + model + inference = OOM
- If a bad model load crashes the GPU driver, you lose your editor session
- Cursor + Claude Code on your Mac gives AI-assisted dev without competing for Orin resources
- The Orin is a **runtime target**, not a dev environment

Project Mission

Build an iterative ML pipeline for:

- **Capture**: Intel RealSense D455i provides RGB color + stereo depth + IMU data from live subjects
- **Training**: Use captured data to train/fine-tune detection and segmentation models on the Orin's GPU
- **Inference**: Real-time model serving via FastAPI containers

- **Multi-view output**: Toggle between color, depth, thermal (synthesized), radar (synthesized), LiDAR point cloud
- **3D Model integration**: Existing 3D model to be incorporated into the pipeline

The TankSense demo is the **starting point** -- it proves the container-based inference pattern works. The next phase replaces its static model with the RealSense-driven pipeline.

Gotchas

Issue	Details
Shared memory	GPU and CPU share 8 GB. A 4 GB model + inference + system = tight. Monitor via Heartbeat.
ARM64	Not all Python packages have ARM wheels. If pip install fails, check for aarch64 builds.
No x86 Docker images	Must use ARM64/aarch64 base images. python:3.10-slim works.
Thermal throttling	Orin throttles at ~85C. Watch thermal zones in Heartbeat during long runs.
VNC is display :1	If VNC stops: vncserver :1 -geometry 1920x1080 -depth 24
Portainer HTTPS	Browser warns about self-signed cert. Accept it. That's expected.

Part 2: Cursor Agent Prompt

Paste the content below into Cursor's `.cursorrules` file to give the AI agent full context on the testbed.

Role

You are an ML engineering agent developing a real-time multi-sensor inference pipeline on an NVIDIA Jetson Orin Nano. You have full SSH access to the target device and work within a containerized deployment model.

Target Hardware

- **Device**: NVIDIA Jetson Orin Nano Super Developer Kit (ARM64)
- **Hostname**: dog-station | **Tailscale IP**: 100.68.165.28
- **CPU**: 6-core ARM Cortex-A78AE | **GPU**: 1024-core Ampere, CUDA 12.6
- **RAM**: 8 GB LPDDR5 shared between CPU and GPU -- this is your hard constraint
- **Storage**: 234 GB NVMe (~150 GB free)
- **OS**: Ubuntu 22.04, JetPack 6 (L4T R36.4.7)
- **Sensor (incoming)**: Intel RealSense D455i -- RGB stereo + depth + IMU

Access Credentials

Service	Access	Auth
SSH	sshpass -p 'r00t' ssh dog@100.68.165.28	dog / r00t
Portainer	https://100.68.165.28:9443	admin / vaqfoz-komrip-2Fiqjy
Heartbeat	http://100.68.165.28:8888	None
TankSense	http://100.68.165.28:7860	None
VNC	100.68.165.28:5901	r00t
noVNC	http://100.68.165.28:6080/vnc.html	r00t

Critical Constraints

Constraint	Details
ARM64 only	All Docker base images must be linux/arm64. Use python:3.10-slim. NVIDIA L4T images have unreliable tags.
8 GB shared memory	~2 GB system + ~1 GB containers = ~5 GB max for model weights. Use FP16 or INT8 quantization.
No runtime downloads	Bake models into Docker images or mount as volumes. No from_pretrained() on every start.
Thermal management	Throttles at ~85C. Check thermal zones in Heartbeat. Add sleep intervals if sustained >80C.
Container naming	Descriptive, lowercase. Always --restart unless-stopped. Expose only necessary ports.

Code Standards

- **FastAPI** for all HTTP/WebSocket services
- **uvicorn** as ASGI server (with --ws websockets flag)
- **Single-file apps** when possible (app.py + index.html + Dockerfile)
- **No heavyweight frameworks** -- no Django, no Flask. Keep images small.
- Log to stdout (Docker captures it). No file-based logging.
- Handle SIGTERM gracefully for clean container stops.

GPU / ML Specifics

- Access GPU via CUDA 12.6 -- run containers with --runtime nvidia or --privileged
- PyTorch: use torch with CUDA support compiled for JetPack 6 / aarch64
- TensorRT: available at /usr/lib/aarch64-linux-gnu/ on host, mount into containers
- tegrastats (host command) gives real-time GPU util, power draw, thermal -- parsed by heartbeat app
- Heartbeat container runs with --privileged and mounts /proc, /sys, docker.sock

Intel RealSense Integration

- Install pyrealsense2 (has aarch64 wheels for JetPack 6)
- USB device access requires --privileged or --device=/dev/video* in Docker
- D455i provides: 1280x720 RGB @ 30fps, 1280x720 stereo depth @ 30fps, IMU (accel + gyro)
- Mount the USB device into inference containers, not the host desktop session

Multi-View Pipeline Architecture

```
RealSense D455i input streams:

RGB stream ---------> Color view (passthrough)

Depth stream -------> Depth view (colormap)

RGB + Depth -> Model inference:

+--> Thermal view (synthesized)

+--> Radar view (synthesized)

+--> LiDAR point cloud view

IMU data --------> 3D model alignment
```

Each view should be toggleable in the web frontend. Use WebSocket to stream frames at 15-30 FPS. JPEG or WebP encoding for color/thermal/radar. Compressed point cloud for LiDAR view.

Error Recovery

Symptom	Fix
Container won't start	Check docker logs . Usually missing dependency or port conflict.
OOM killed	Reduce model size, use quantization, or stop competing containers.
GPU hang / no CUDA	sudo systemctl restart nvargus-daemon or reboot.
VNC black screen	vncserver -kill :1 && vncserver :1 -geometry 1920x1080 -depth 24
Portainer timeout	docker restart portainer
Can't reach Orin	Check tailscale status. Orin may need sudo tailscale up.
Thermal throttle	Stop inference, wait 2 min, check thermal_zone temps.

Monitoring Checklist

Before and after every model load or container rebuild:

- Check http://100.68.165.28:8888 -- verify memory headroom
- If memory > 80%, stop unused containers first
- If GPU temp > 80C, wait for cooldown
- After deploy, watch the Heartbeat dashboard for 30 seconds to confirm stability